Apache Kafka

Document ID

DOC012

Project

Apache Kafka

Category

Data Engineering

Batch

batch_4

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Kafka Overview

Apache Kafka is a distributed event streaming platform used to move and retain streams of records between producers and consumers. Instead of requiring one application to call another application directly, producers write events to Kafka topics and consumers read those events independently. This decoupling is useful in data engineering when many downstream systems need access to the same stream of business events.

Topics, Partitions, and Offsets

Kafka stores events inside topics. Topics are divided into partitions so data can be distributed and processed in parallel. Within a partition, each record receives an offset that identifies its position in the ordered log. Consumers track offsets to know which records have already been processed. Partitioning improves scalability, but the chosen partition key also influences ordering because Kafka guarantees ordering within a partition rather than across an entire topic.

Producers and Consumers

A producer publishes events to a topic, while a consumer reads events from one or more topic partitions. Consumer groups allow multiple consumers to share the work for a topic. Each partition is assigned to only one consumer within a consumer group at a time, which allows parallel processing while avoiding duplicate work inside that group. Different consumer groups can independently read the same events for different use cases.

Streaming Pipelines and Delivery Semantics

Kafka is often used as the transport layer for near-real-time pipelines. A downstream application may consume events, transform them, and write results to a database, data lake, or another topic. Reliable pipeline design requires careful handling of failures, retries, and offsets. At-least-once processing can cause a record to be handled more than once, so downstream operations should be idempotent when duplicates would be harmful.

Retention and Replay

Kafka retains events for a configured period rather than deleting them immediately after consumption. This makes replay possible. A consumer can reset its offset and process older events again, which is useful for rebuilding derived data or recovering from a bug. Replay is powerful, but the downstream pipeline must be designed to handle repeated historical events safely and consistently.